ReturnFlow

A deployable, policy-grounded apparel return assistant

DSAN 6700 — Final Project, Deliverable 2: Midpoint Check-in

Team: Zejun (Derry) Xu *Complete team roster pending confirmation.*

Date: October 23, 2026

GitHub: `https://github.com/Zejun-Derry-Xu/DML_Project`

Jetstream2 application URL: Pending instructor-access acceptance

Working draft, not submission-ready. The candidate has a recorded Jetstream2 deployment, but its normal UI is accessible through an SSH tunnel, not an instructor-facing public URL. Replace the URL above only after external access is tested. Confirm all team names before submission. No current VM uptime is asserted by this document.

1 Problem, users and midpoint scope

Customers often describe a return incompletely: they mention fit, but omit the order, item condition or tag status. A conventional form asks every question up front; an unconstrained chatbot may invent missing facts or make inconsistent policy decisions. ReturnFlow separates **collecting a customer's statements** from **authorizing a simulated return**. It guides customers through only the missing information, explains the applicable policy, and routes unresolved cases to a durable simulated support ticket.

What a user can do

Choose a fictional account, describe an apparel return, select an owned order/item, answer condition questions with cards or text, review the result, and explicitly confirm an eligible simulated return. A separate policy-help panel answers general questions without modifying the active return. A refreshed page can recover the case.

Engineering priorities

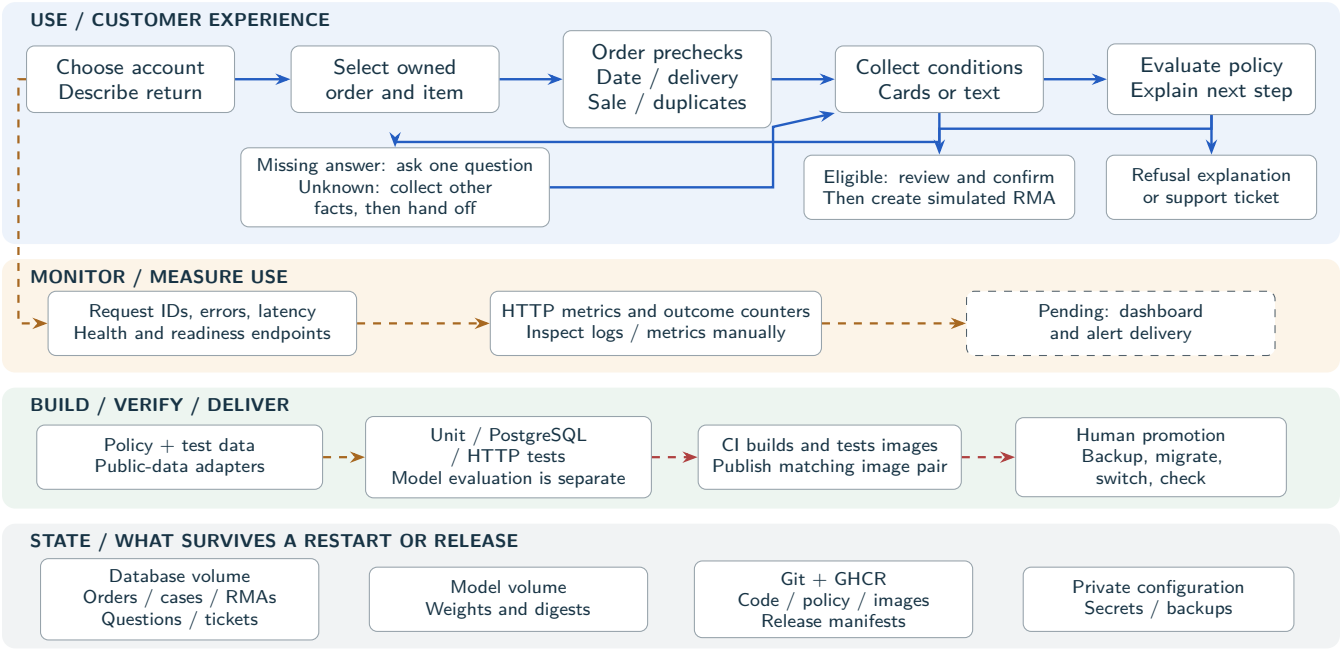
- **Reliable deployment first:** one small application stack, reproducible images, bounded model resources, version identity, backups and visible failure states.
- **A replaceable AI boundary:** Qwen extracts candidate facts; application code owns account/order checks, missing-information handling, policy decisions and confirmation.
- **Explicit uncertainty:** an omitted condition is not a negative answer, and a model's valid JSON is not proof of a true statement.

Checkpoint status	What the evidence supports
Deployed candidate	Release `847c7b3`: frontend, API and PostgreSQL; guided cards; policy explanations; recorded HTTP and persistence acceptance.
Implemented / CI-tested	Source snapshot `073f31b`: delivery tooling, automated checks and opt-in extraction-harness experiments. Newer source is not automatically the deployed release.
Experimental / pending	Qwen 4B/8B trials are not formal production qualification. Public access, a real two-release rollback rehearsal and enforced merge checks remain open.

Prototype boundary. All customers, orders, tickets and return authorizations are simulated. There is no real payment, refund, shipment, staffed help desk or production login.

2 Solution architecture

The diagram connects the user journey to development, monitoring and the artifacts that persist. Solid arrows are request/workflow paths; dashed arrows are measurement or release feedback. Proposed services are explicitly marked as pending.



Process: build, use and monitor

The midpoint focuses on a complete small user journey, not on adding agents or training a model. Development uses synthetic policy cases and public-data adapters; automated checks protect the application path. Operational metrics support inspection, while automated alerts and a dashboard remain planned.

Execution: questions are normal API responses

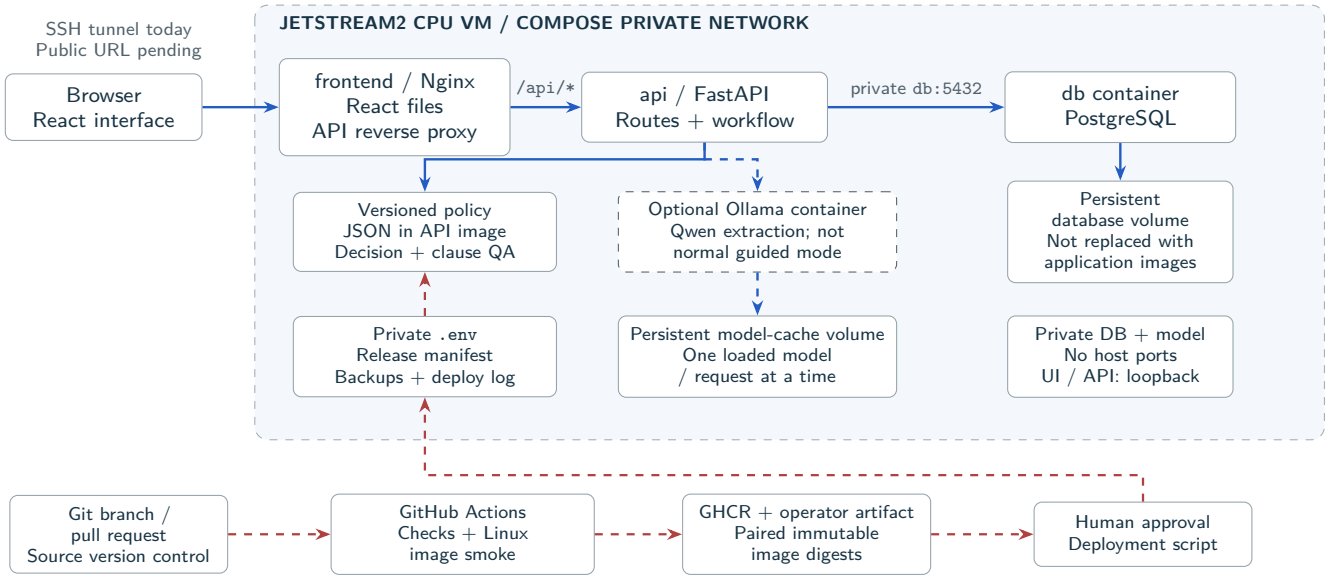
The backend returns the next question and approved choices as part of its response. The browser renders them. The customer's next button click or text message starts a new request; the server does not keep a connection waiting for an answer.

State: uncertainty and final actions are durable

Known, false, missing and conflicting observations are not interchangeable. Case state and active questions persist in PostgreSQL. Support tickets are internal records, not notifications to a real agent. A simulated RMA requires an explicit confirmation and a duplicate-return check; refreshing the browser must not create another one.

3 Technical architecture and deployment boundary

ReturnFlow uses a **modular monolith**: one FastAPI process with separate route, workflow, extraction, policy and persistence modules. It avoids the network and coordination overhead of multiple agent services. The target is a CPU-only Jetstream2 VM; no GPU is required for the installed guided prototype.



Containers and communication

The installed guided stack has **three containers**: frontend, API and database. A qualified model deployment would add Ollama as a fourth service. The API addresses PostgreSQL and Ollama by Compose service DNS; container `localhost` is not the host or another container. Qdrant is not needed for the installed lexical/extractive policy path, although a vector-retrieval adapter exists as an optional extension.

Configuration, persistence and restart behavior

Deployment uses `compose.jetstream.yml`, private environment settings, named data/model volumes, health checks and `restart: unless-stopped`. The restart policy restarts an exited process; an unhealthy flag alone does not imply automatic repair. CPU/memory caps limit contention. Only synthetic data are permitted: the fictional-account selector is not production authentication.

Version identity

The API and frontend images carry the source commit. A release manifest binds their digests, schema expectation and policy checksum. Readiness checks database tables and columns as well as application identity. A responding model endpoint or a green `/ready` is not evidence that real model inference is accurate or fast enough.

4 Prototype interaction, API and core analytic

A representative customer interaction

1. Choose a fictional account and enter: *"The jacket is too small; I would like to return it."* The interface requests selection of an owned order/item.
2. The API checks trusted order data: delivery, the 30-day return window, final-sale status and an existing return. A failed precheck produces a policy-based explanation rather than unnecessary condition questions.
3. For an eligible order, collect the raw return reason and the missing resale conditions: stains, garment damage, attached tags and intact tags. The installed guided mode uses explicit choices; an enabled LLM may interpret natural language.
4. If a condition remains unknown, collect the remaining information and create a persistent simulated support ticket. Otherwise evaluate the versioned policy and show an eligibility result or refusal with relevant clauses.
5. Only an eligible case with explicit confirmation creates a simulated RMA. Reopening the case returns its stored state. No money is transferred.

This is an **illustrative walkthrough**, not a new measured test result. Recorded cloud acceptance separately exercised order ownership, unknown-condition handoff, ticket persistence/resume and cross-account denial.

Representative API surface

HTTP endpoint	Responsibility
`GET /api/v1/demo/customers` `POST /api/v1/demo/session`	Select a fictional account; issue a signed demo cookie. This is a demo identity mechanism, not real authentication.
`POST /api/v1/chat`	Submit new text; return case state, explanation and the next question.
`POST /api/v1/questions/{id}/answer`	Submit a card choice or permitted free-text answer; the server validates question ownership and state.
`GET /api/v1/return-cases/{session}`	Restore the authorized customer's case, question, ticket or simulated return.
`POST /api/v1/policy/ask`	Retrieve policy clauses and provide a cited explanation; do not mutate the active return.
`GET /health`, `/ready`, `/metrics`	Operational probes, version/schema readiness and Prometheus-format application metrics. FastAPI exposes API docs at `/docs`.

Core analytic: language, policy and retrieval have different jobs

The semantic component extracts only customer-stated candidate facts with evidence. It does not decide eligibility. The configured policy requires appropriate resale conditions; wearing alone does not cause refusal. Customer-caused adverse conditions can be ineligible; arrival issues, defects and uncertain responsibility require review.

The current apparel policy is `apparel-1.0.0.json`. It supplies decision parameters and rendered explanation clauses from one versioned source. Policy help uses lexical retrieval and extractive answers in the installed configuration. This is not a claim that an LLM or vector search authorizes returns. A policy change still needs tests and a release; a new kind of policy may require application changes.

Model boundary. Exact evidence checks and structured JSON prevent some mechanical errors, not all semantic mistakes. Wrong-item references, unsupported values, negation and instructions inside user text remain known risks. Qwen trials and harness experiments are not formal qualification or a claim of production readiness.

5 Continuous delivery, testing and recovery

Build once; promote deliberately

GitHub Actions checks Python lint/types/tests, runs database migrations against real PostgreSQL, exercises API workflows, and lints/builds the frontend. It then builds Linux AMD64 API/frontend images, starts those exact images with an isolated database, and performs HTTP smoke. An approved release trigger publishes the *same* tested images to GHCR and uploads an operator bundle; there is no second build on the VM.

Main pushes and explicitly authorized candidate tags can publish. Ordinary experiment pushes only validate. The manually published candidate `847c7b3` passed CI validation, image publication and artifact upload. The newer `073f31b` snapshot also passed CI; that does not mean it was deployed. **This is continuous delivery with manual promotion, not automatic continuous deployment.**

What smoke tests establish

Layer	Checks and limits
CI image smoke	Health/readiness, workflow/schema/commit identity, a known policy citation, and a guided account/order/cards/confirmation path to a simulated RMA. Tests use an isolated disposable database; they do not establish Qwen accuracy.
Deployment smoke	Read-only application health, expected identity, policy citation and matching frontend/API versions. It does not create a live return merely to test uptime.
Model acceptance	Separate serial inference/evaluation and a real-model application workflow check. Downloading weights, seeing a model name or passing guided smoke does not satisfy this gate. Formal qualification remains incomplete.

Manual deployment sequence

1. Validate the release manifest and immutable image pair; acquire a deployment lock. Pull images and check commit labels before switching the application.
2. Stop API/frontend writes, take a PostgreSQL dump and verify archive readability. Apply additive migrations using the new image; preserve existing volumes and records.
3. Start the selected image pair without rebuilding. Check frontend/API commit, schema expectation, policy checksum and read-only smoke; record success only afterward.

Failures stop the process with stage-specific errors. Some failures leave the app stopped or a pending pair running; the runbook describes inspection and recovery. No automatic database overwrite or destructive downgrade is used as a repair shortcut.

Persistent data makes rollback different from code checkout

The cloud adoption included a database backup and restoration into a separate test database, then additive migration from schema `0002` to `0004`, preserving historical cases and the existing simulated return. Archive listing alone would not prove recovery, so the distinct restore check matters.

A rollback checker and unit tests exist, but a **real two-release compatibility rehearsal is pending**. The deployment script refuses an unapproved old image pair. Rolling back code must retain new business records and a compatible expanded schema; it must not mean deleting a volume or restoring an old dump over new data.

Current governance gap. CI reports pass/fail, but main has no enforced required-check rule in the last repository inspection. Branch protection, formal version releases/change notes and the actual rollback rehearsal are next delivery tasks.

6 Evaluation, limitations and next steps

Evaluate the deployed system, not just a model score

Dimension	Current approach	Limit / next evidence
Functional correctness	Unit, API, PostgreSQL and image-smoke checks; ownership, persistence, unknown handoff and duplicate/confirmation safeguards.	Isolated tests do not prove high availability or all real-world return behavior.
Extraction / retrieval	Public ABCD adapters and authored apparel diagnostics; score labelled fields, unsupported fills and policy citations separately.	Unlabelled fields are not gold negatives. Authored labels are provisional; tau2 end-to-end benchmarking remains future work.
Efficiency / usability	Track request latency/errors and outcome counters. Guided cards give a usable no-model path; only missing facts are requested.	Measure actual completion time, extra questions, abandonment and user ease of use; no user study claimed.
Cost / capacity	CPU VM, small Compose stack, resource-limited optional model, single model request concurrency and reusable cached weights.	No billed-cost saving, throughput SLA or concurrent-load capacity claim. Cloud and Mac timings are not interchangeable.

Known deployment limitations

- **Access:** the instructor-facing application URL is pending. The recorded installation is loopback-only; `localhost` on an instructor's laptop is not our VM. Only an application entry point should be made reachable, not database/model ports.
- **Reliability:** one VM, no replicas or automatic failover; upgrades have planned downtime. Local backups need off-host retention. Metrics are exposed but no deployed monitoring/alerting platform is claimed.
- **Security and scope:** fictional accounts and synthetic orders only. No production identity provider, real refund integration or physical-condition inspection.
- **Model operation:** CPU cold starts/timeouts and unsupported facts remain risks. Model experimentation is paused in favor of delivery work; the normal guided prototype is not presented as a qualified Qwen-powered production service.

Finish the checkpoint without expanding the architecture

1. Confirm the team roster and instructor access method; reopen the VM only when needed, verify a meaningful input/output path and replace the application URL on page 1.
2. Refresh deployment/diagram evidence against the exact selected release. Clearly label installed, experimental and planned components; retain the guided/model distinction.
3. Add required CI checks for main, complete a disposable two-version rollback exercise, and add versioned release notes. These strengthen delivery without new agents, training, Kubernetes or automatic production promotion.

Representative code and operator references

Concern	Repository reference (links pinned to source snapshot)
UI / workflow	`frontend/src/ApparelApp.tsx`; `app/services/apparel_service.py`
Analytic / policy	`app/extractors/apparel_llm.py`; `app/policy/apparel.py`
Delivery / verification	`.github/workflows/ci.yml`; `scripts/deploy_release.py`; `scripts/smoke_apparel.py`
Operator handoff	`docs/apparel-runbook.md`; `scripts/check_rollback.py`